

Software Development Lifecycle für KI-Systeme

Leitfaden mit Ausrichtung auf Spec-Driven Development (SDD)

Fassung 2026 · Projektumsetzung durchgängig mit Gradio · iludis.de

0. Worum geht es	2
1. Der große Rahmen: Der Software-Lebenszyklus (SDLC)	3
2. Zwei Lebenszyklen in einem Projekt	4
3. Die sechs Lerneinheiten im Überblick	5
4. Die 7 Blöcke der Software-Erstellung	6
6. Anforderungsanalyse	13
7. Qualität: ISO 25010 (Fassung 2023)	14
8. Testen	15
9. Deployment & Wartung mit Gradio	16
10. Der rote Faden: Spec-Driven Development (SDD)	17
11. Beispiel-Anforderungen mit Ampel-Markierung	19
Anhang A: Diagramm-Vorlagen (PlantUML, Gradio-Fassung)	20
Anhang B: Links & Material	21

0. Worum geht es

Mittels eines MNIST-Ziffernerkennungs-Projekts einmal den kompletten Lebenszyklus einer Software durchspielen. In jedem Block wächst eine Spezifikation mit. Am Ende des Kurses haben die Studierenden nicht nur eine App gebaut, sondern auch erlebt, wie man Software zuerst beschreibt und dann baut — das ist die Grundidee von Spec-Driven Development.

Legende: Die Ampel-Kästen

Durch das ganze Skript ziehen sich farbige Kästen. Sie beantworten immer dieselbe Frage: Kann man diesen Teil des Projekts vorab in einer Spezifikation festschreiben — oder nicht?

MIT SDD ABBILDBAR

Grün: Das lässt sich vorab genau aufschreiben und später prüfen. Es gehört in die Spezifikation.

NICHT MIT SDD ABBILDBAR

Rot: Das lässt sich nicht vorab festschreiben. Es ist Experimentierarbeit, Statistik oder laufender Betrieb. SDD kann hier höchstens den Rahmen setzen.

TEILWEISE MIT SDD ABBILDBAR

Gelb: Grenzfall. Ein Teil davon ist spezifizierbar, ein Teil nicht.

1. Der große Rahmen: Der Software-Lebenszyklus (SDLC)

Software entsteht nicht in einem Rutsch. Sie durchläuft Phasen — so wie ein Haus nicht mit der ersten Mauer beginnt, sondern mit einem Gespräch darüber, wer darin wohnen soll. Die klassischen Phasen sind:

- **Anforderungsanalyse:** Was soll das System können? Für wen?
- **Entwurf / Architektur:** Wie bauen wir es? Aus welchen Teilen besteht es?
- **Implementierung:** Der eigentliche Bau — Code schreiben.
- **Test:** Tut das System, was wir versprochen haben?
- **Deployment:** Das System für Nutzer verfügbar machen.
- **Wartung:** Beobachten, pflegen, verbessern — der längste Abschnitt im Leben einer Software.

Wasserfall vs. Agil: Beim Wasserfall-Modell werden die Phasen einmal der Reihe nach abgearbeitet — wie beim Hausbau: Erst der komplette Plan, dann der Bau. Beim agilen Vorgehen läuft man in kleinen Runden immer wieder durch alle Phasen — wie bei einem Garten: pflanzen, schauen, umplanen, nachpflanzen. Unser Kurs läuft aus Zeitgründen einmal linear durch alle Phasen (also eher Wasserfall). Echte Projekte laufen heute fast immer in Runden.

MIT SDD ABBILDBAR — Warum SDD hier gut andockt

Jede SDLC-Phase erzeugt ein Zwischenergebnis: User Stories, ein Architekturbild, ein Testkonzept. SDD fasst diese Zwischenergebnisse zu einem einzigen wachsenden Dokument zusammen: der Spezifikation. Sie ist die eine Quelle der Wahrheit — für Menschen und für KI-Assistenten, die beim Bauen helfen.

2. Zwei Lebenszyklen in einem Projekt

Ein KI-System besteht aus zwei sehr unterschiedlichen Teilen. Der eine Teil ist **klassische Software**: Oberfläche, Abläufe, Schnittstellen. Ihr Verhalten wird **programmiert** — jemand schreibt hin, was passieren soll. Der andere Teil ist das **Modell**: Sein Verhalten wird **gelernt** — es entsteht aus Daten.

Deshalb hat das Modell seinen eigenen Kreislauf¹: Daten sammeln → Daten aufbereiten → Modell trainieren → Modell bewerten → Modell einsetzen → im Betrieb beobachten → mit neuen Daten neu trainieren. Dieser Kreislauf hört nie auf, denn die Welt ändert sich, und das Modell bleibt stehen, wenn niemand nachlegt.

Das ist die zentrale Botschaft dieses Kapitels: Beide Kreisläufe laufen parallel und treffen sich an einer klar definierten Stelle — der Schnittstelle zum Modell.

MIT SDD ABBILDBAR — Der Vertrag an der Modellgrenze

Was in das Modell hineingeht (ein 28×28-Graustufenbild), was herauskommt (eine Ziffer 0–9 plus Wahrscheinlichkeiten) und wie gut es mindestens sein muss (z. B. Trefferquote $\geq 95\%$ auf dem Testdatensatz) — das alles lässt sich vorab festschreiben. Diese Schnittstelle ist ein Vertrag und gehört in die Spezifikation.

NICHT MIT SDD ABBILDBAR — Das Innere des Modells

Wie das Modell zu seinen Entscheidungen kommt, welche Merkmale es intern gelernt hat, wie es auf ein einzelnes, nie gesehenes Bild reagiert — das kann keine Spezifikation garantieren. Das Modell bleibt hinter dem Vertrag eine Wahrscheinlichkeits-Maschine. Genauso wenig planbar: Experimentierarbeit wie Datenexploration oder die Suche nach guten Trainingseinstellungen. Hier weiß man das Ergebnis erst hinterher.

¹Als formalisiertes Prozessmodell hierfür ist CRISP-DM (Cross-Industry Standard Process for Data Mining) verbreitet; für den Kurs genügt der Kreislauf in vereinfachter Form.

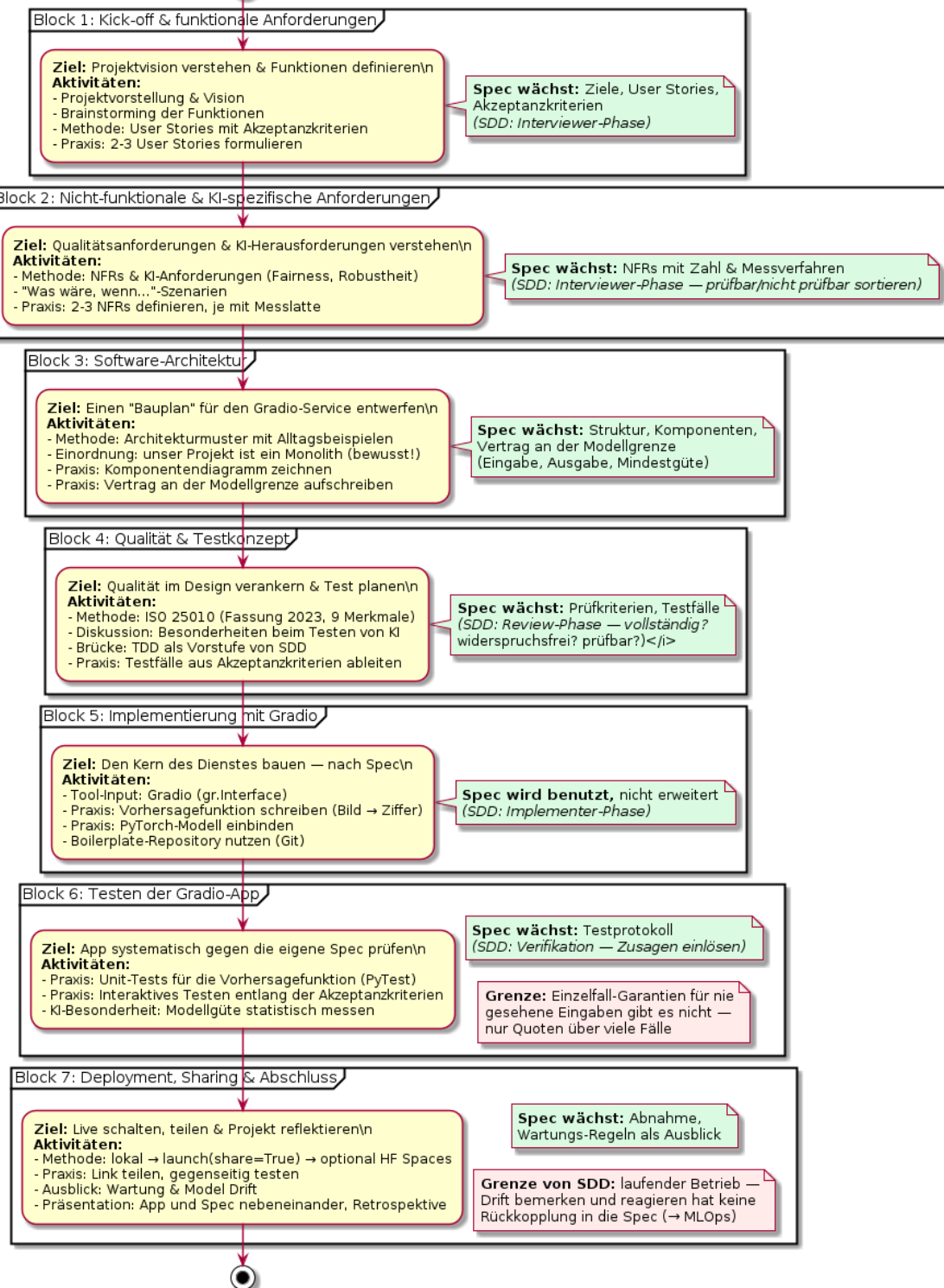
3. Die sechs Lerneinheiten im Überblick

Die inhaltliche Grundstruktur des alten Skripts bleibt erhalten. Die Formulierungen sind vereinfacht, die Brücken zwischen den Einheiten auf Gradio und SDD angepasst.

Einheit	Kerninhalt und Ziel	Brücke zur nächsten Einheit
1. Der große Rahmen	Software entsteht in Phasen (SDLC). Die Studierenden können alle späteren Aktivitäten in ein Gesamtbild einordnen.	Jeder Prozess beginnt mit einer Idee. Wie wird aus einer Idee eine konkrete Anforderung?
2. Das Fundament	Anforderungen erheben: verstehen, was Nutzer wirklich brauchen. Funktionale und nicht-funktionale Anforderungen unterscheiden, User Stories schreiben.	Was, wenn das Problem zu komplex für feste Regeln ist? Hier kommt KI ins Spiel.
3. Die KI-Perspektive	Wann ist KI sinnvoll? Was braucht sie (Daten!), und welche besonderen Anforderungen bringt sie mit (Fairness, Erklärbarkeit)? Dazu neu: die zwei Lebenszyklen aus Kapitel 2.	Wir wissen, was wir bauen wollen. Wie sieht der Bauplan aus?
4. Der Bauplan	Software-Architektur: Systeme in Teile gliedern und das Modell sauber einbinden. Architekturbilder lesen und selbst zeichnen.	Ein Bauplan existiert. Aber was macht einen guten Bauplan aus? Welche Qualität wollen wir?
5. Der Maßstab	Qualität ist messbar: ISO 25010 (Fassung 2023) plus KI-spezifische Qualität. Abwägungen treffen und begründen.	Wir haben Qualitätsziele. Wie prüfen wir, ob wir sie erreichen?
6. Die Prüfung	Testen: Teststufen, Black-Box/White-Box, und die Besonderheit bei KI — es gibt oft kein einzelnes „richtiges“ Ergebnis. Systematisches Misstrauen als Denkweise.	Der Kreis schließt sich: Testergebnisse führen zu Änderungen — zurück an den Anfang des Zyklus.

4. Die 7 Blöcke der Software-Erstellung

Block	Inhalt	Spec wächst um ...
1	Kick-off und funktionale Anforderungen: Projektvision, Brainstorming, User Stories	Ziele, User Stories
2	Nicht-funktionale und KI-spezifische Anforderungen: NFRs, „Was wäre, wenn ...“-Szenarien	NFRs, Messlatten
3	Architektur: Bauplan des Gradio-Service, Komponentendiagramm, Vertrag an der Modellgrenze	Struktur, Vertrag
4	Qualität und Testkonzept: ISO 25010:2023, KI-Qualität, Testfälle aus Akzeptanzkriterien	Prüfkriterien
5	Implementierung mit Gradio: Vorhersagefunktion, gr.Interface, Modell einbinden	— (Spec wird benutzt)
6	Testen der Gradio-App: Unit-Tests für die Kernlogik, interaktives Testen der Oberfläche	Testprotokoll
7	Deployment und Sharing mit Gradio, Ausblick Wartung, Präsentation und Retrospektive	Abnahme, Ausblick



Block 1: Kick-off & funktionale Anforderungen

Ziel: Die Projektvision verstehen und die grundlegenden Funktionen definieren.

- Vorstellung des Projekts: „Wir bauen einen Web-Dienst, der handgeschriebene Ziffern erkennt — auf Basis eures vorhandenen MNIST-Modells.“
- Brainstorming: Was muss der Dienst können? (Bild entgegennehmen, Ziffer zurückgeben, ...)
- Methoden-Input: User Stories („Als ... möchte ich ... damit ...“).
- Praxis: 2–3 User Stories formulieren, jede mit mindestens einem Akzeptanzkriterium.

MIT SDD ABBILDBAR

User Stories mit Akzeptanzkriterien sind der Kern jeder Spezifikation. Beispiel: „Als Nutzerin möchte ich ein Bild hochladen, um die Ziffer zu erfahren. Akzeptanz: Nach dem Klick auf Erkennen erscheint innerhalb von 2 Sekunden eine Ziffer von 0 bis 9.“ Das ist präzise, prüfbar — grün.

Block 2: Nicht-funktionale & KI-spezifische Anforderungen

Ziel: Verstehen, dass es neben dem Was auch ein Wie gut gibt — und dass KI eigene Anforderungen mitbringt.

- Methoden-Input: Nicht-funktionale Anforderungen (NFRs) und KI-spezifische Anforderungen (Fairness, Erklärbarkeit, Robustheit).
- Szenario Performance: „Was wäre, wenn der Dienst 1.000 Anfragen pro Sekunde bedienen müsste?“
- Szenario Robustheit: „Was wäre, wenn jemand das Modell mit absichtlich falschen Bildern austrickst?“
- Praxis: 2–3 wichtige NFRs für das Projekt definieren — jede mit einer Zahl, wo es geht.
-

TEILWEISE MIT SDD ABBILDBAR — NFRs: mal grün, mal rot

Grüner Anteil: „95 % der Anfragen werden in unter 2 Sekunden beantwortet“ — messbar, prüfbar, spezifizierbar. Roter Anteil: „Die Oberfläche soll sich gut anfühlen“ — ehrliches Ziel, aber kein Prüfstein. Solche Wünsche dürfen in der Spec stehen, sie müssen aber als nicht prüfbar gekennzeichnet werden. **Die NFRs in prüfbar/nicht prüfbar sortieren.**

Block 3: Software-Architektur

Ziel: Einen Bauplan für den Gradio-Dienst entwerfen.

- Methoden-Input: Architekturmuster ohne Vorwissen — siehe Kapitel 5. Unser Projekt wird dabei ehrlich eingeordnet: Es ist ein Monolith.
- Diskussion: Welche Teile hat unser Dienst? (Oberfläche, Vorhersagefunktion, Modelldatei)
- Praxis: Ein einfaches Komponentendiagramm für den MNIST-Gradio-Dienst zeichnen (Vorlage im Anhang).
- Praxis: Den Vertrag an der Modellgrenze aufschreiben: Eingabe, Ausgabe, Mindestgüte.

MIT SDD ABBILDBAR

Die Struktur eines Systems — welche Teile es gibt, wer mit wem spricht, über welche Schnittstellen — ist voll spezifizierbar. Genau das zeigt ein Komponentendiagramm. In SDD-Sprache: Die Verdrahtung ist Regie-Wissen, die einzelnen Teile sind Bausteine.

Block 4: Qualität & Testkonzept

Ziel: Qualität als planbare, messbare Größe verstehen und daraus Testfälle ableiten.

- Methoden-Input: Qualitätsmerkmale nach ISO 25010:2023 (Kapitel 7) und ihre Bedeutung für unser Projekt.
- Diskussion: Wie testet man ein KI-System? Was ist überhaupt ein „richtiges“ Ergebnis?
- Praxis: Ein einfaches Testkonzept erstellen — welche Testfälle folgen aus den User Stories und NFRs? (Funktionstest: Bild rein → Ziffer raus; KI-Test: bekanntes Bild → bekannte Ziffer?)
- Brücke zu SDD: Testgetriebene Entwicklung (TDD) heißt „erst der Test, dann der Code“. SDD geht einen Schritt weiter: „erst die Spezifikation, dann alles andere“. TDD ist der prüfbare Kern von SDD.

Block 5: Implementierung mit Gradio

Ziel: Den Kern des Dienstes bauen — und dabei erleben, dass die Spec die Arbeit trägt.

- Tool-Input: Kurzeinführung in Gradio (gr.Interface: Funktion + Eingabetyp + Ausgabebetyp = fertige Web-App).
- Praxis: Eine Python-Funktion für die Vorhersage schreiben (Bild → Ziffer), das trainierte PyTorch-Modell einbinden.
- Praxis: Das Gradio-Interface bauen: Zeichenfeld und Bild-Upload als Eingaben, erkannte Ziffer plus Wahrscheinlichkeiten als Ausgabe.
- Boilerplate-Repository bereitstellen (Git), damit niemand Zeit mit dem Setup verliert.

Block 6: Testen der Gradio-App

Ziel: Die Kernlogik und die Oberfläche systematisch prüfen — gegen die eigene Spezifikation.

- Methode: Unit-Tests für die Vorhersagefunktion (bekanntes Bild → erwartete Ziffer, kaputtes Bild → saubere Fehlermeldung).
- Praxis: Manuelles, interaktives Testen der Web-Oberfläche entlang der Akzeptanzkriterien aus Block 1.
- KI-Besonderheit: Die Modellgüte wird nicht mit Einzelfällen „bewiesen“, sondern statistisch auf dem Testdatensatz gemessen (Kapitel 8).

MIT SDD ABBILDBAR

Jeder Testfall, der aus einem Akzeptanzkriterium folgt, ist grün: Er stand sinngemäß schon in der Spec, bevor eine Zeile Code existierte.

NICHT MIT SDD ABBILDBAR

„Erkennt das Modell auch die krakelige 4 meines Sitznachbarn?“ — für einzelne, nie gesehene Eingaben gibt es keine Garantie. Prüfbar ist nur die Quote über viele Fälle, nie der Einzelfall. Diese Grenze gehört explizit in den Unterricht.

Block 7: Deployment, Sharing & Abschluss

Ziel: Den Dienst live schalten, teilen — und den Blick auf das richten, was nach dem Go-Live kommt.

- Methode: Deployment-Optionen mit Gradio — lokal starten, per share=True einen öffentlichen Link erzeugen, optional dauerhaft auf Hugging Face Spaces veröffentlichen.
- Praxis: App starten, Link im Kurs teilen, gegenseitig testen.
- Ausblick Wartung: Was passiert, wenn sich die Handschriften der Nutzer ändern? (Model Drift — Kapitel 9)
- Präsentation und Retrospektive: Jede Gruppe zeigt App und Spezifikation nebeneinander. Leitfrage: Wo hat die Spec geholfen, wo hat sie gefehlt?

5. Architekturmodelle — ohne Vorwissen erklärt

Architektur beantwortet eine einfache Frage: Aus welchen Teilen besteht ein System, und wie reden diese Teile miteinander? Jedes Muster unten wird nach demselben Schema erklärt: die Idee in einem Satz, ein Beispiel aus dem Alltag, ein Beispiel aus der Software-Welt — und der Bezug zu unserem Projekt.

KOMMENTAR ZUR ÜBERARBEITUNG

Im alten Skript wurden die Begriffe weitgehend als bekannt vorausgesetzt und die Beispiele waren reine Namenslisten (Netflix, Uber, SAP ...). Neu: Jeder Begriff wird über ein Alltagsbild eingeführt, bevor Software-Beispiele kommen. Außerdem wird jedes Muster auf unser MNIST-Gradio-Projekt bezogen — das fehlte vorher völlig.

5.1 Monolith

Idee in einem Satz: Alles steckt in einem einzigen Programm — eine Codebasis, ein Start, ein Ganzes.

Alltagsbeispiel: Ein Imbisswagen. Eine Person nimmt die Bestellung an, brät, kassiert und gibt aus. Alles an einem Ort, alles aus einer Hand. Das ist schnell aufgebaut und leicht zu überblicken. Aber: Wenn die Person krank ist, steht der ganze Wagen. Und wenn plötzlich 200 Leute anstehen, kann man nicht einfach „nur die Kasse“ verdoppeln — man braucht einen zweiten kompletten Wagen.

Software-Beispiele: WordPress (das komplette Redaktionssystem ist eine PHP-Anwendung), eine klassische Desktop-Anwendung wie ein Bildbearbeitungsprogramm — und: ein Jupyter-Notebook, in dem Datenaufbereitung, Training und Auswertung untereinander stehen.

Unser Projekt: Die MNIST-Gradio-App ist ein Monolith. Oberfläche, Vorhersagefunktion und Modell laufen in einem einzigen Python-Prozess. Für unseren Zweck ist das genau richtig: ein Entwickler-Team (die Gruppe), ein überschaubarer Funktionsumfang, kein Skalierungsdruck. Monolith ist kein Schimpfwort — es ist oft die vernünftigste Wahl.

5.2 Client-Server

Idee in einem Satz: Einer fragt (Client), einer antwortet (Server) — Arbeitsteilung über ein Netzwerk.

Alltagsbeispiel: Ein Restaurant. Der Gast (Client) bestellt beim Kellner, die Küche (Server) bereitet zu und liefert. Viele Gäste, eine Küche. Vorteil: Die Küche hat alles zentral im Griff — Rezepte, Vorräte, Qualität. Nachteil: Wenn die Küche brennt, bekommt kein Gast mehr etwas; und bei zu vielen Gästen wird die Küche zum Engpass.

Software-Beispiele: Webbrowser und Webserver, eine App und ihre Datenbank, ein E-Mail-Programm und der Mail-Server.

Unser Projekt: Nach außen ist unsere App Client-Server: Der Browser der Nutzerin ist der Client, die laufende Gradio-App der Server. Wichtig für den Unterricht: Ein System kann aus mehreren Blickwinkeln verschiedene Muster zeigen — innen Monolith, nach außen Client-Server. Das ist kein Widerspruch.

5.3 Schichtenarchitektur (Layered Architecture)

Idee in einem Satz: Das System ist in Etagen gegliedert; jede Etage hat eine Aufgabe und redet nur mit ihren Nachbar-Etagen.

Alltagsbeispiel: Eine Behörde. Vorne der Schalter (Kontakt zum Bürger), dahinter die Sachbearbeitung (die eigentlichen Entscheidungen), im Keller das Archiv (die Akten). Der Bürger geht nie selbst ins Archiv — er redet mit dem Schalter, der Schalter mit der Sachbearbeitung, die Sachbearbeitung mit dem Archiv. Vorteil: klare Zuständigkeiten. Nachteil: Manchmal wird ein Zettel nur durchgereicht, ohne dass eine Etage etwas damit tut — unnötiger Laufweg.

Software-Übersetzung: Präsentationsschicht (Oberfläche) → Logikschicht (Verarbeitung) → Datenschicht (Speicherung). Beispiele: Django-Web-Apps (Templates → Views → Models) oder klassische Banksoftware.

Unser Projekt: Auch unser kleiner Monolith hat innen Schichten: das Gradio-Interface (Präsentation), die Vorhersagefunktion (Logik) und die Modelldatei model.pt (Daten). Gute Übung: Die Studierenden zeichnen diese drei Schichten und markieren, welche Schicht welche kennt.

5.4 Microservices

Idee in einem Satz: Statt eines großen Programms viele kleine, unabhängige Dienste, die über Schnittstellen (APIs) miteinander reden.

Alltagsbeispiel: Ein Foodcourt statt eines Restaurants. Viele kleine Stände: einer macht Nudeln, einer Burger, einer Getränke. Jeder Stand hat eigenes Personal, eigene Kasse, eigenes Lager (= eigene Datenbank). Fällt der Burger-Stand aus, gibt es trotzdem Nudeln. Wird der Nudel-Stand überrannt, stellt man einen zweiten Nudel-Stand daneben — ohne die anderen anzufassen. Der Preis: Man braucht Wegweiser, Regeln und Absprachen zwischen den Ständen. Ein Foodcourt ist deutlich aufwendiger zu organisieren als ein Imbisswagen.

Software-Beispiele: Netflix (getrennte Dienste für Videoabruf, Empfehlungen, Nutzerkonten, Abrechnung), große Online-Shops (Katalog, Warenkorb, Bezahlung, Versand als eigene Dienste). Kommunikation typischerweise über REST-APIs oder Nachrichten-Warteschlangen.

5.5 Serviceorientierte Architektur (SOA)

Idee in einem Satz: Der ältere Verwandte der Microservices: Auch hier Dienste, aber größer geschnitten, und der gesamte Nachrichtenverkehr läuft über eine zentrale Vermittlungsstelle (Enterprise Service Bus).

Alltagsbeispiel: Eine Firmen-Poststelle: Keine Abteilung schickt direkt an eine andere, alles geht über die Poststelle, die sortiert, übersetzt und weiterleitet. Vorteil: einheitliche Regeln, gute Integration alter und neuer Abteilungen. Nachteil: Ist die Poststelle überlastet oder fällt aus, steht die ganze Firma.

Für den Kurs: SOA nur als historische Einordnung erwähnen — der zentrale Vermittler ist der wesentliche Unterschied zu Microservices, wo die Dienste direkt miteinander reden.

5.6 Event-Driven Architecture (EDA)

Idee in einem Satz: Komponenten rufen sich nicht direkt an, sondern hängen Nachrichten (Events) an ein schwarzes Brett — wer zuständig ist, reagiert.

Alltagsbeispiel: Ein Gruppenchat im Sportverein. Jemand schreibt „Platz 2 ist heute gesperrt“. Der Absender ruft nicht jeden Einzelnen an — er weiß nicht einmal, wer die Nachricht braucht. Die Trainerin verlegt daraufhin ihr Training, der Platzwart plant um, alle anderen lesen nur mit. Absender und Empfänger sind entkoppelt.

Software-Beispiele: IoT-Systeme (Temperatursensor meldet einen Wert, die Klimaanlage reagiert), Bestellabwicklung in Shops über Nachrichten-Warteschlangen wie Kafka oder RabbitMQ.

Unser Projekt: Kommt bei uns nicht vor — als Gedankenexperiment aber nützlich: „Was, wenn jede erkannte Ziffer als Event an ein Statistik-Modul gemeldet würde?“

5.7 Woran man Architekturen unterscheidet — fünf Fragen

- **Aufteilung:** Alles in einem (Monolith), viele kleine Dienste (Microservices) oder Etagen (Schichten)?
- **Datenhaltung:** Eine gemeinsame Datenbank oder jeder Dienst seine eigene?
- **Kommunikation:** Direkte Funktionsaufrufe, Schnittstellen (APIs) oder Nachrichten (Events)?
- **Bereitstellung:** Alles zusammen ausliefern oder jedes Teil einzeln?
- **Zuständigkeit:** Hat jedes Teil genau eine Aufgabe — oder überlappen sich die Aufgaben?

MIT SDD ABBILDBAR — Architektur ist Spec-Land

Die Antworten auf diese fünf Fragen lassen sich vollständig vorab aufschreiben und prüfen. Ein Komponentendiagramm ist nichts anderes als der visuelle Teil der Spezifikation. Wer hier präzise ist, macht die Implementierung — für Menschen wie für KI-Assistenten — fast zum Abschreiben.

6. Anforderungsanalyse

Funktionale Anforderungen sagen, was das System tun soll: „Der Dienst nimmt ein Bild entgegen und gibt eine Ziffer zurück.“ **Nicht-funktionale Anforderungen (NFRs)** sagen, wie gut es das tun soll: wie schnell, wie sicher, wie verständlich, wie zuverlässig.

Werkzeuge für funktionale Anforderungen

- **User Stories:** „Als ... möchte ich ... damit ...“ — eine Anforderung aus Sicht einer Person.
- **Akzeptanzkriterien:** Woran erkennen wir, dass die Story erfüllt ist? Immer prüfbar formulieren.
- **Use-Case-Diagramme:** Wer nutzt das System wofür? (Vorlage im Anhang.)

Typische NFR-Kategorien

- Performance (Antwortzeit, Durchsatz) · Zuverlässigkeit und Verfügbarkeit · Sicherheit und Datenschutz
- Benutzerfreundlichkeit · Wartbarkeit

Priorisieren mit MoSCoW

Must have: ohne das kein Start · **Should have:** wichtig, aber nicht kritisch · **Could have:** schön zu haben · **Won't have:** bewusst ausgeschlossen. Gerade das W ist wertvoll: Was wir nicht bauen, gehört genauso in die Spezifikation wie das, was wir bauen.

Wie man an Anforderungen kommt

- Interviews mit Beteiligten (wer nutzt das System, wer bezahlt es, wen betrifft es?)
- Workshops mit Priorisierung (z. B. MoSCoW gemeinsam ausfüllen)
- Prototypen und Mockups: Ein Bild sagt mehr als zehn Meetings
- Reviews: Anforderungen in Runden verfeinern und von den Beteiligten abnehmen lassen

MIT SDD ABBILDBAR

In SDD-Sprache ist die Anforderungserhebung die Interviewer-Phase: Durch gezieltes Fragen („Was wäre, wenn ...?“) wird aus einer vagen Idee eine präzise, prüfbare Spezifikation. Die „Was wäre, wenn“-Szenarien aus Block 2 sind genau solche Interviewer-Fragen.

NICHT MIT SDD ABBILDBAR

Die Gesprächsführung selbst — Vertrauen aufbauen, Zwischentöne hören, Zielkonflikte zwischen Beteiligten moderieren — ist Handwerk und Menschenkenntnis. Das Ergebnis landet in der Spec; der Weg dorthin lässt sich nicht spezifizieren.

7. Qualität: ISO 25010 (Fassung 2023)

Qualität ist kein Bauchgefühl, sondern messbar. Die Norm ISO/IEC 25010:2023² nennt neun Merkmale von Produktqualität:

Merkmal (2023)	Leitfrage — einfach gestellt
Functional Suitability	Tut das System das Richtige — vollständig und korrekt?
Performance Efficiency	Ist es schnell und sparsam genug (Zeit, Speicher, Ressourcen)?
Compatibility	Verträgt es sich mit anderen Systemen und kann Daten austauschen?
Interaction Capability (früher: Usability)	Kommen Menschen gut damit zurecht — lernbar, bedienbar, barrierefrei?
Reliability	Läuft es stabil und erholt sich von Fehlern?
Security	Sind Daten und Zugänge geschützt (Vertraulichkeit, Integrität)?
Safety (neu 2023)	Kann das System Menschen oder Umwelt gefährden — und ist das abgesichert?
Maintainability	Lässt es sich gut verstehen, ändern und testen?
Flexibility (früher: Portability)	Lässt es sich an neue Umgebungen und Anforderungen anpassen?

KI-spezifische Qualität — zusätzlich zur Norm

- **Fairness:** Behandelt das Modell alle Gruppen gleich gut? (Bei MNIST greifbar: Wird die 1 genauso zuverlässig erkannt wie die 8? Erkennt es europäische und amerikanische Schreibweisen der 7?)
- **Erklärbarkeit:** Können wir sagen, warum das Modell so entschieden hat?
- **Robustheit:** Was passiert bei verrauschten, gedrehten oder absichtlich manipulierten Eingaben?
- **Unsicherheit:** Sagt das Modell auch, wie sicher es sich ist — und ist diese Angabe ehrlich?

Begriffshygiene: Die Konfusionsmatrix ist keine Metrik, sondern eine Tabelle, aus der man Metriken ableitet — etwa Genauigkeit, Precision und Recall. Im alten Skript stand sie unter „Metriken“; die Unterscheidung lohnt sich im Unterricht.

TEILWEISE MIT SDD ABBILDBAR — Qualität in der Spec

Grüner Anteil: Jede Qualitätsanforderung mit Zahl und Messverfahren — „Trefferquote $\geq 95\%$ auf dem MNIST-Testdatensatz“, „Antwort in unter 2 Sekunden“ — ist eine vollwertige, prüfbare Spec-Zusage.

Roter Anteil: Die Prüfung von Fairness und Robustheit ist Statistik über viele Fälle, keine Struktur-Prüfung. Die Spec kann die Messlatte festlegen, aber nicht garantieren, dass das Modell sie reißt oder schafft — das zeigt erst das Experiment.

²ISO/IEC 25010:2023, Systems and software engineering — SQuARE — Product quality model.

8. Testen

Denkweise: systematisches Misstrauen. Nicht nur den Schönwetter-Fall prüfen („Happy Path“), sondern gezielt Ränder und Ausnahmen: leere Eingaben, riesige Bilder, Buchstaben statt Ziffern.

Teststufen — von klein nach groß

- **Unit-Test:** ein einzelner Baustein, z. B. die Vorhersagefunktion allein.
- **Integrationstest:** spielen die Bausteine zusammen? (Oberfläche ruft Vorhersage korrekt auf?)
- **Systemtest:** die ganze App aus Nutzersicht, im Browser.
- **Abnahmetest:** erfüllt die App die Akzeptanzkriterien aus der Spezifikation?

Black-Box und White-Box

Black-Box: Man kennt nur Eingabe und Ausgabe und prüft von außen — wie ein Getränkeautomat: Geld rein, Getränk raus, Innenleben egal. White-Box: Man schaut in den Code und prüft gezielt die inneren Wege — wie ein Techniker, der den Automaten öffnet.

Besonderheiten beim Testen von KI

Bei klassischer Software gibt es zu jeder Eingabe ein festes Soll-Ergebnis. Beim Modell nicht: Es gibt kein „richtig“ für jede denkbare Krakelschrift. Deshalb testet man KI **statistisch** (Güte über einen ganzen Testdatensatz) und mit zwei besonderen Tricks:

- **Adversarial Testing:** absichtlich fiese Eingaben bauen (Rauschen, Verzerrung), um Schwächen zu finden — der Crashtest fürs Modell.
- **Metamorphic Testing:** Man kennt das richtige Ergebnis nicht, aber man kennt Beziehungen: Ein leicht verschobenes Bild derselben 7 sollte immer noch als 7 erkannt werden. Ändert sich die Antwort, stimmt etwas nicht.

MIT SDD ABBILDBAR — TDD als Brücke zu SDD

Testgetriebene Entwicklung (TDD) heißt: erst den Test schreiben, dann den Code, der ihn besteht. Der Test ist damit eine ausführbare Mini-Spezifikation. SDD verallgemeinert das: Die gesamte Spezifikation — Struktur, Verhalten, Messlatten — entsteht vor dem Code. Alles Prüfbare aus der Spec wird zu Testfällen; Block 6 ist dann nur noch das Einlösen dieser Zusagen.

NICHT MIT SDD ABBILDBAR

Nicht abbildbar: die Suche nach guten Testdaten für Adversarial-Fälle und die Interpretation statistischer Ergebnisse („Sind 94,7 % gut genug, wenn wir 95 % versprochen haben?“). Das ist Urteilkraft, keine Spezifikation.

9. Deployment & Wartung mit Gradio

Drei Stufen der Bereitstellung

- **Lokal:** `demo.launch()` — die App läuft auf dem eigenen Rechner, erreichbar im Browser unter `localhost`.
- **Geteilt:** `demo.launch(share=True)` — Gradio erzeugt einen öffentlichen Link, der 72 Stunden gültig ist. Perfekt für die Kurs-Situation: Link in den Chat, alle testen gegenseitig. Wichtig zu besprechen: Der eigene Rechner ist dabei der Server — Klappe zu, App weg.
- **Dauerhaft:** Hugging Face Spaces — die App liegt auf einem fremden Server und läuft auch, wenn der eigene Rechner aus ist. Optional als Zusatzaufgabe für schnelle Gruppen.

Wartung: Was nach dem Go-Live passiert

Model Drift, einfach erklärt: Die Welt ändert sich, das Modell bleibt stehen. Wenn unsere App plötzlich von Menschen genutzt wird, die Ziffern anders schreiben als die Menschen im Trainingsdatensatz, sinkt die Trefferquote — ohne dass sich am Code irgendetwas geändert hat. Die Antwort darauf ist der Kreislauf aus Kapitel 2: beobachten, neue Daten sammeln, neu trainieren.

NICHT MIT SDD ABBILDBAR — Die Grenze von SDD

Der laufende Betrieb ist die härteste Grenze der Methode. Eine Spezifikation beschreibt ein Soll — sie merkt aber nicht von selbst, wenn die Wirklichkeit davonläuft. Klassisches SDD hat keine eingebaute Rückkopplung aus dem Betrieb.

Was die Spec leisten kann: Wartungs-Regeln vorab festschreiben („Trefferquote wird monatlich auf frischen Stichproben gemessen; fällt sie unter 93 %, wird neu trainiert“). Was sie nicht leisten kann: das Messen, Bemerkern und Reagieren selbst. Ein ehrlicher Schlusspunkt für den Kurs — und ein Ausblick: In echten ML-Teams heißt dieses Feld MLOps.

10. Der rote Faden: Spec-Driven Development (SDD)

Die Idee in einem Satz: Erst wird genau aufgeschrieben, was gebaut werden soll — so genau, dass ein Mensch oder ein KI-Assistent danach bauen kann. Dieses Dokument heißt Spezifikation (kurz: Spec), und es ist die einzige Quelle der Wahrheit im Projekt.

Warum jetzt? KI-Assistenten können heute erstaunlich gut Code schreiben — aber nur so gut, wie man ihnen sagt, was sie bauen sollen. Eine vage Anweisung erzeugt vagen Code. Eine präzise Spezifikation dagegen macht die Implementierung zum am besten kontrollierbaren Schritt des ganzen Projekts. Die wertvollste Fähigkeit verschiebt sich damit vom Codeschreiben zum Präzise-Beschreiben — und genau das üben die Studierenden in diesem Kurs, Block für Block.

Wie die Spec durch den Kurs wächst

Block	Beitrag zur Spezifikation	SDD-Sicht
1	Ziele, User Stories, Akzeptanzkriterien	Interviewer-Phase: Fragen stellen, präzisieren
2	NFRs mit Messlatten, KI-Anforderungen	Interviewer-Phase: „Was wäre, wenn ...?“
3	Systemstruktur, Komponenten, Vertrag an der Modellgrenze	Struktur-Teil der Spec
4	Qualitätskriterien, Testfälle	Review-Phase: Ist die Spec vollständig, widerspruchsfrei, prüfbar?
5	— (die Spec wird benutzt, nicht erweitert)	Implementer-Phase: Bauen nach Spec
6	Testprotokoll gegen die Spec	Verifikation: Zusagen einlösen
7	Abnahme, Wartungs-Regeln als Ausblick	Grenze der Methode sichtbar machen

Ein Prüf-Trick für Block 4: Vor der Implementierung geht die Gruppe ihre Spec mit drei Fragen durch — Ist jede Anforderung prüfbar formuliert? Widersprechen sich zwei Anforderungen? Fehlt zu einer User Story ein Akzeptanzkriterium? Das ist ein Review im Kleinen und lässt sich gut als Partnerarbeit zwischen zwei Gruppen organisieren: Jede Gruppe prüft die Spec der anderen.

Die Landkarte: Was ist grün, was ist rot?

Das Gesamtbild des Kurses in einer Tabelle — sie eignet sich auch als Abschluss-Folie:

Projektbestandteil	Ampel	Warum
Funktionale Anforderungen mit Akzeptanzkriterien	GRÜN	präzise formulierbar, prüfbar
Messbare NFRs (Zeit, Durchsatz, Verfügbarkeit)	GRÜN	Zahl + Messverfahren = prüfbar
Architektur, Komponenten, Schnittstellen	GRÜN	Struktur ist vorab festlegbar
Vertrag an der Modellgrenze (Ein-/Ausgabe, Mindestgüte)	GRÜN	Schnittstelle + Messlatte
Testfälle aus Akzeptanzkriterien	GRÜN	folgen direkt aus der Spec
Qualitätsziele mit Metrik (Fairness-Schwelle, Robustheits-Quote)	GELB	Messlatte grün, Nachweis ist Statistik
Weiche NFRs („fühlt sich gut an“)	GELB	darf in die Spec, ist aber nicht prüfbar
Inneres Verhalten des Modells, Einzelfall-Garantien	ROT	gelernt, nicht programmiert
Datenexploration, Feature-Suche, Trainings-Tuning	ROT	Ergebnis erst hinterher bekannt
Gesprächsführung, Moderation, Urteilstkraft	ROT	Handwerk, keine Spezifikation
Laufender Betrieb, Drift bemerken und reagieren	ROT	Spec hat keine Rückkopplung aus der Wirklichkeit

Merksatz für die Studierenden: SDD spezifiziert die Hülle und den Vertrag — nicht den Kern und nicht den Weg. Die deterministische Software drumherum ist grün, das gelernte Modell dahinter ist rot, und der Vertrag an der Grenze ist das, was beide Welten zusammenhält.

11. Beispiel-Anforderungen mit Ampel-Markierung

Die beiden ausgearbeiteten Beispiele aus dem alten Skript (California Housing und Kreditkarten-Betrugserkennung) bleiben als Übungsmaterial erhalten — hier kompakt und mit Ampel-Markierung. Sie eignen sich als Analyse-Übung: Die Studierenden markieren selbst, was grün, gelb oder rot ist, bevor sie diese Auflösung sehen.

Beispiel 1: Immobilienpreis-Vorhersage (Regression)

Anforderung	Ampel	Begründung
Vorhersage in unter 2 Sekunden, Ausgabe: Preis in USD mit Konfidenzintervall	GRÜN	Schnittstelle + Zeit, beides prüfbar
Mittlerer Fehler (MAE) unter 50.000 \$	GRÜN	Messlatte mit Verfahren
Stapelverarbeitung: 1.000 Immobilien in unter 5 Minuten	GRÜN	prüfbarer Durchsatz
„Wichtigste Preisfaktoren werden erklärt“	GELB	dass erklärt wird: prüfbar — wie gut: kaum
Keine Benachteiligung einzelner Stadtteile	GELB	als Metrik-Schwelle grün, Nachweis statistisch
Nutzerzufriedenheit über 4,2 von 5 Sternen	ROT	Geschäftsziel, kein Systemverhalten
Welche Merkmale das Modell am Ende nutzt	ROT	Ergebnis des Experiments, nicht planbar

Beispiel 2: Kreditkarten-Betrugserkennung (Klassifikation)

Anforderung	Ampel	Begründung
Antwortzeit unter 100 ms für 99 % der Anfragen	GRÜN	harte, messbare Zusage
Ausgabe: Risiko-Wert 0–100 plus Einstufung plus Empfehlung	GRÜN	Schnittstellen-Vertrag
Precision über 0,9 und Recall über 0,8 auf dem Prüfdatensatz	GRÜN	Messlatte mit Verfahren
Transaktionsdaten nach spätestens 90 Tagen löschen	GRÜN	Regel, automatisiert prüfbar
Jede Entscheidung wird begründet (Erklärbarkeit)	GELB	Vorhandensein prüfbar, Güte kaum
Modell bleibt wirksam, obwohl Betrüger ihre Muster ändern	ROT	Drift — nur durch Betrieb + Retraining beherrschbar
60 % weniger Betrugsfälle im ersten Jahr	ROT	Geschäftsziel mit vielen Fremdeinflüssen

Anhang A: Diagramm-Vorlagen (PlantUML, Gradio-Fassung)

Zum Rendern: <https://editor.plantuml.com/uml/> — alternativ draw.io für freies Zeichnen. Die kodierten Diagramm-Links des alten Skripts sind entfernt; die Quelltexte unten genügen.

Use-Case-Diagramm

```
@startuml
left to right direction
actor "Nutzer" as user
rectangle "Gradio KI-Service" {
    usecase "Ziffer ins Zeichenfeld malen" as UC1
    usecase "Bild zur Erkennung hochladen" as UC2
    usecase "Ergebnis (Ziffer) ansehen" as UC3
    usecase "Service über Link teilen" as UC4
}
user -- UC1
user -- UC2
user -- UC3
user -- UC4
@enduml
```

Komponentendiagramm

```
@startuml MNIST_Gradio
actor Benutzer
component "Gradio-Interface\n(Oberfläche)" as UI
component "Vorhersagefunktion\n(Python, PyTorch)" as Predict
database "Trainiertes Modell\n(model.pt)" as Model
Benutzer --> UI : Bild malen / hochladen (Browser)
UI --> Predict : ruft auf (Funktionsaufruf)
Predict --> Model : lädt beim Start
@enduml
```

Systemkontextdiagramm

```
@startuml
actor "Nutzer" as user
rectangle "Gradio KI-Service" as system
[Webbrowser] as browser
[Gradio Sharing-Dienst] as sharing
user -up-> browser : "bedient"
browser -right-> system : "Bild hochladen / interagieren"
system -left-> browser : "zeigt interaktive UI"
system -down-> sharing : "registriert sich für öffentlichen Link"
sharing -left-> user : "stellt Link bereit"
@enduml
```

Aktivitätsdiagramm

```
@startuml
|Nutzer|
start
:Öffnet die Web-Anwendung;
while (Weitere Ziffer erkennen?) is (ja)
    if (Eingabemethode?) then (malen)
        :Malt eine Ziffer in das Zeichenfeld;
    else (hochladen)
        :Wählt eine Bilddatei aus;
    endif
:Klickt auf "Erkennen";
end
```

```
|System|
:Verarbeitet das Eingabebild;
:Führt Vorhersage mit dem Modell aus;
:Stellt Ergebnis bereit;
|Nutzer|
:Sieht die erkannte Ziffer;
endwhile (nein)
:Schließt die Anwendung;
stop
@enduml
```

Anhang B: Links & Material

- Use-Case-Vorlagen: online.visual-paradigm.com
- Quiz SDLC-Grundlagen: claude.ai/public/artifacts/120f66b4...
- Quiz Use Cases: claude.ai/public/artifacts/4600bf65...
- Datensatz Übung 1: [California Housing \(Kaggle\)](#)
- Datensatz Übung 2: [Credit Card Fraud \(Kaggle\)](#)